

Poprawa efektywności metody wstecznej propagacji błędu

Algorytm wstecznej propagacji błędu

1. Wygeneruj losowo wektory wag.
2. Podaj wybrany wzorzec na wejście sieci.
3. Wyznacz odpowiedzi wszystkich neuronów wyjściowych sieci:

$$y_k^{wyj} = f\left(\sum_{j=1}^I w_{kj}^{wyj} y_j^{wyj-1}\right)$$

4. Oblicz błędy wszystkich neuronów warstwy wyjściowej:

$$\delta_k^{wyj} = z_k - y_k^{wyj}$$

5. Oblicz błędy w warstwach ukrytych (pamiętając, że, aby wyznaczyć błąd w warstwie $h - 1$, konieczna jest znajomość błędu w warstwie po niej następującej - h):

$$\delta_j^{h-1} = \frac{df(u_j^{h-1})}{du_j^{h-1}} \sum_{k=1}^I \delta_k^h w_{kj}^h$$

6. Zmodyfikuj wagi wg zależności:

$$w_{ji}^{h-1} = w_{ji}^{h-1} + \eta \delta_j^{h-1} y_i^{h-1}$$

7. Wróć do punktu 2.

Wady backpropagationa

- ✓ Nie można zagwarantować, iż proces uczenia doprowadzi do odnalezienia minimum globalnego funkcji miary błędu - często zdarza, że odnalezione zostaje minimum lokalne,
 - Wybranie niewłaściwego punktu startowego czyli niewłaściwy dobór wartości początkowych wag oraz nieodpowiedniej drogi może spowodować wejście w minimum lokalne, którego algorytm nie będzie w stanie opuścić.
- ✓ Funkcja miary błędu jest funkcją wielokrotnie symetryczną w wielowymiarowej przestrzeni wag, co powoduje występowanie wielu minimów globalnych i lokalnych.
- ✓ Klasyczna metoda wstecznej propagacji błędów wymaga dużej liczby iteracji by osiągnąć zbieżność oraz jest wrażliwa na występowanie minimów lokalnych.
- ✓ Podstawowy algorytm BP może się okazać **zbyt wolny**, jeżeli przyjmie się **za mały współczynnik uczenia**, z kolei **zbyt duża wartość współczynnika** grozi wystąpieniem oscylacji.

Poprawa efektywności metody BP

- ✓ Na poprawę efektywności procesu uczenia może mieć wpływ wiele czynników:
 - wagi początkowe - nie mogą być zbyt duże; **zaleca się losowy wybór wag**, by sygnał wyjściowy nieliniowej części neuronu był nieco mniejszy od jedności.
 - **korzystnie wpływa kilkukrotne powtarzanie uczenia**, rozpoczynające się od różnych wartości wag.
 - kolejność podawania wektorów uczących - **zaleca się podawanie wektorów wzorcowych w losowej kolejności**.
- ✓ Przedstawione powyżej propozycje mogą w pewnym stopniu zapobiec problemom pojawiającym się w trakcie uczenia sieci. Najlepszym rozwiązaniem jest dokonanie modyfikacji samego algorytmu wstecznej propagacji błędów tak, aby przyspieszyć zbieżność procesu uczenia oraz uniknąć minimów lokalnych.

Wsteczna propagacja błędów z momentum

- ✓ Jednym z rozwiązań umożliwiających bezpieczne zwiększenie efektywnego tempa uczenia bez pogarszania stabilności procesu jest zastosowanie momentowej metody wstecznej propagacji błędów - MBP (ang. Momentum BackPropagation).

Metoda BP z momentum

- ✓ Istotą metody jest wprowadzenie do procesu uaktualniania wagi pewnej bezwładności tzw. "momentu", proporcjonalnego do zmiany tej wagi w poprzedniej iteracji:

$$w_{ji}(t) = w_{ji}(t-1) + \eta \delta_j(t-1) y_i(t-1) + \mu (w_{ji}(t-1) - w_{ji}(t-2))$$

Momentum

- $\mu \in (-0,1]$ - współczynnik momentu.
- Dwa pierwsze składniki po prawej stronie wyrażenia są identyczne, jak w zwykłej metodzie BP, natomiast ostatni uwzględnia poprzednią zmianę wagi i jest niezależny od aktualnej wartości gradientu.

- ✓ Składowik momentu wpływa pozytywnie w sytuacjach gdy:
 - występują duże płaskie odcinki funkcji błędu,
 - kolejne gradienty są przeciwnie skierowane,
 - uczenie „przebiega” przez minimum lokalne funkcji błędu,
 - występują na powierzchni funkcji błędu tzw. wąwozy (wąskie obszary o stromych ścianach bocznych i głębokich dnach). Zmiany gradientu o „wysokiej częstotliwości” (oscylacje w poprzek ścian wąwozu) są eliminowane, a wzmacniany jest składowik gradientu wymuszający ruch w stronę dna.
- ✓ Składowik momentu nie powinien zdominować procesu uczenia, ponieważ grozi to jego niestabilnością - aby zapobiec temu kontroluje się wartość funkcji błędu w trakcie uczenia, doprowadzając do jej wzrostu jedynie w ograniczonym zakresie np. o k procent.

Modyfikacje BP ze zmiennymi współczynnikami uczenia i momentu

- ✓ Właściwy dobór współczynnika uczenia η oraz współczynnika momentu μ ma duży wpływ na zbieżność procesu uczenia.
- ✓ Optymalne wartości współczynników uczenia i momentu mogą być różne dla różnych iteracji, a nawet dla każdej z wag danej iteracji.
- ✓ Wartości współczynników uczenia i momentu przyjęte na początku procesu uczenia sieci mogą okazać się niewłaściwe. Znacznie zmniejsza to efektywność uczenia, a w skrajnym przypadku prowadzi do rozbieżności procesu uczenia.
- ✓ Rozwiązanie tego problemu może być zmiana współczynnika uczenia i momentu w trakcie procesu uczenia sieci.

Techniki zmian współczynników uczenia i momentum:

- ✓ zastosowanie większych wartości η oraz μ na początku uczenia i zmniejszanie ich w miarę zbliżania się funkcji błędu do minimum,
- ✓ przyjęci na początku procesu uczenia małych wartości współczynnika uczenia η (ewentualnie również współczynnika momentu μ). Gdy proces uczenia nabierze tempa współczynniki są zwiększane, by w końcowej fazie ponownie ich zmniejszyć,
- ✓ uzależnienie współczynnika uczenia od wartości funkcji błędu średniokwadratowego – gdy błąd maleje można zwiększyć współczynnik uczenia, gdy rośnie należy go zmniejszyć. Badania wykazują, że dobrze jest zwiększać współczynnik uczenia o wartość stałą, a zmniejszać go geometrycznie.

Metoda przyspieszonej wstecznej propagacji

Quickprop

- ✓ Algorytm został opracowany przez S.E. Fahlmana w 1988
- ✓ W metodzie tej zakłada się, że funkcja błędu jest lokalnie **paraboloidalna**
- ✓ Algorytm modyfikacji wag przebiega następująco:

$$\Delta w_{ji}(t) = -\eta(k) \left[\frac{\partial E(w(k))}{\partial w_{ij}} + \gamma w_{ij}(k) \right] + \mu_{ij}(k) (w_{ji}(t-1) - w_{ji}(t-2))$$

- współczynnik γ prowadzi do zmniejszania wag, nie pozwalając na ich zbyt duży wzrost. Typowe wartości są niewielkie.
 - Współczynnik uczenia η może przyjmować dwie wartości: wartość stałą η na starcie uczenia albo wartość zerową.
 - Współczynnik momentu μ dostosowuje się adaptacyjnie do aktualnych postępów w uczeniu.
- ✓ Algorytm Quickprop powoduje duże przyspieszenie procesu uczenia.
 - ✓ Zabezpiecza on przed utknięciem w płytkim minimum lokalnym

Algorytm *RPROP*

- ✓ Algorytm został opracowany przez M. Riedmillera i H. Brauna (Riedmillera, Brauna 1992).
- ✓ Jego nazwa pochodzi od nazwy angielskiej Resilient backPROPagation.
- ✓ Istotą metody jest uwzględnienie w procesie aktualizacji wagi tylko znaku składowej gradientu (jej wartość jest pomijana):

$$\Delta w_{ji}(t) = -\theta \operatorname{sgn}\left(\frac{\partial E(w(k))}{\partial w_{ij}}\right)$$

- Współczynnik uczenia θ jest uzależniony od zmian wartości gradientu. Jeśli w obu kolejnych iteracjach znak gradientu jest taki sam następuje wzrost θ , w przeciwnym przypadku zachodzi jego redukcja.
- ✓ Algorytm RPROP powoduje znaczne przyspieszenie procesu uczenia zwłaszcza w obszarach o niewielkim nachyleniu funkcji błędu.

Metoda Newtona

- ✓ Alternatywne podejście do zagadnienia minimalizacji funkcji błędu polega na próbie osiągnięcia położonego najniżej punktu powierzchni błędu
w jednym kroku - co jest osiągalne przy założeniu, że **znany** jest kształt jej powierzchni.
- ✓ Przyjmując, że wektor $\mathbf{w}$ jest wektorem wszystkich wag (w całej sieci) można wyrazić wartość gradientu funkcji kryterium w otoczeniu pewnego punktu $\mathbf{w}_0$ w postaci szeregu Taylora:

$$\nabla E(\mathbf{w}) = \nabla E(\mathbf{w}_0) + \mathbf{H}(\mathbf{w} - \mathbf{w}_0) + \dots$$

- gdzie $\mathbf{H}$ jest macierzą drugich pochodnych (hesjanem) funkcji błędu E .
 - dalszych wyrazów szeregu nie uwzględniamy, zakładając, że funkcja błędu ma kształt funkcji kwadratowej (paraboloidy).
- ✓ W punkcie $\mathbf{w}$, w którym funkcja $E(\mathbf{w})$ osiąga minimum wartość gradientu wynosi 0, przeto właśnie

$$\mathbf{w} = \mathbf{w}_0 - \mathbf{H}^{-1} \nabla E(\mathbf{w})$$

- $\mathbf{H}$ – hesjan funkcji celu
- ∇ - gradient funkcji celu

- ✓ W praktyce funkcja błędu prawie nigdy nie ma kształtu paraboloidy, dlatego zwykle nie udaje się trafić w punkt rzeczywistego minimum „jednym strzałem”, oznacza to konieczność iterowania rozwiązania zgodnie ze wzorem:

$$\mathbf{w}(t) = \mathbf{w}(t-1) - \eta \mathbf{H}^{-1} \nabla E(\mathbf{w}(t-1))$$

- ✓ Metoda ta jest bardzo kosztowna numerycznie, ponieważ w każdym kroku stosowania algorytmu należy odwrócić macierz drugich pochodnych.
- ✓ metoda Newtona bywa **niestabilna numerycznie**, zwłaszcza w przypadku, gdy punkt startowy nie jest położony dostatecznie blisko poszukiwanego rozwiązania

Algorytm Levenberga - Marquardta (LM)

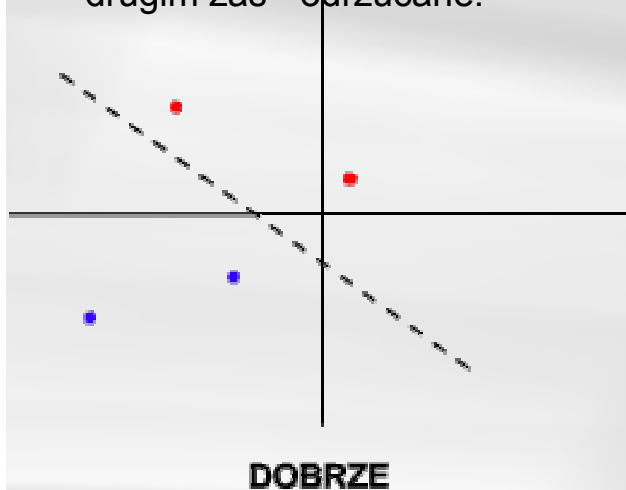
- ✓ Jest on jednym z najbardziej efektywnych algorytmów do uczenia sieci jednokierunkowych.
- ✓ Łączy w sobie zbieżność algorytmu Gaussa - Newtona blisko minimum, z metodą najszybszego spadku, która bardzo szybko zmniejsza błąd, gdy rozwiązanie jest dalekie.
- ✓ Wzór opisujący ten algorytm wygląda następująco:

$$\Delta \mathbf{w}(t) = -(\mathbf{H} + \lambda \mathbf{I})^{-1} \nabla E(\mathbf{w}(t-1))$$

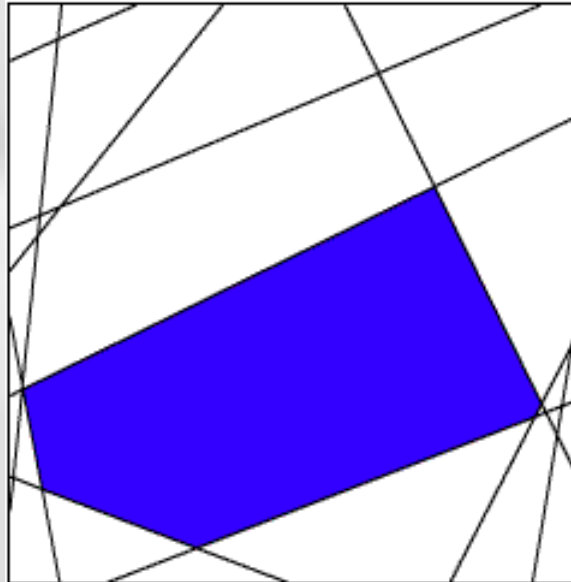
- paramet Marquardta λ zmniejsza się podczas uczenia do 0
 - λ duże (daleko od minimum) – metoda największego spadku
 - λ małe (blisko od minimum) – metoda Newtona
- H – hesjan funkcji celu
- I – macierz jednostkowa

Warstwy sieci nieliniowej

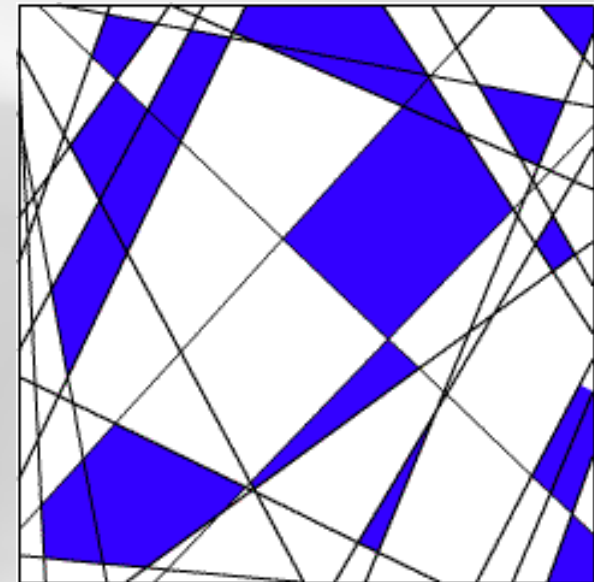
Sieć jednowarstwowa tworzy w przestrzeni wejść linię prostą, która dzieli ową przestrzeń na dwa obszary. W jednym z nich są punkty reprezentujące obiekty akceptowane przez neuron, w drugim zaś - odrzucane.



Sieć dwuwarstwowa wyznacza w przestrzeni wejść wypukły i spójny obszar, w którym znajdują się punkty odpowiadające akceptowanym przez sieć obiektom wejściowym



Sieć trójwarstwowej – wyznacza taki obszar pozytywnej odpowiedzi, który nie musi być ani spójny, ani wypukły



- Sieć trójwarstwowa może rozwiązać każdy rodzaj zadania.
- Różnice między sieciami o różnej liczbie warstw występują tylko w sieciach nieliniowych. Sieć liniowa zawsze, niezależnie od ilości warstw, dzieli obszar przestrzeń wejść linią prostą.

Rozmiary warstw sieci

- ✓ W warstwie wejściowej liczba neuronów musi być równa długości wektora uczącego (ilości podawanych na sieć jednocześnie sygnałów)
- ✓ Liczba neuronów w warstwie wyjściowej musi być równa ilości rozróżnialnych przez sieć klas
- ✓ Ilości neuronów w warstwie ukrytej nie można precyzyjnie określić!

$$l = \sqrt{nm}$$

$$l = \log_2 S$$

s – liczba rozróżnianych klas

n – ilość wejść sieci

m – ilość wyjść sieci